

Kill the Quote Spreadsheet

Product & design note · AI Procurement Analyst prototype

Problem

A category buyer sends an RFQ for 30 line items to five suppliers. Nine days later the replies arrive as a spreadsheet, a PDF, a Word document, a plain email and a photograph of a printed rate card. Three days go into retyping it into Excel. Then one question — *cheapest per line, but only among suppliers who cleared quality* — costs a fourth.

The re-keying is the visible cost. The real one is that the comparison ends up in a spreadsheet nobody can audit, so the reasoning behind a six-figure decision evaporates.

Users

- **The category buyer** — owns the RFQ, does the re-keying today. Needs one comparable view, and a follow-up answered without rebuilding the sheet.
- **The approver** — asks the question that costs the fourth day. Needs the answer *defensible* more than fast: who was considered, who excluded, on what basis.
- **Whoever opens this in six months** — a dispute, an audit, or the same buy again. Needs to see why a line went where it went, without asking whoever decided.

What I'm building

One continuous flow, requirement to executed award. A co-pilot that asks only what materially affects price. Suppliers reply in any format. The system reads every reply, matches it to the buyer's lines, normalises what is safe to normalise, and lands it in one comparison with the evidence attached. The buyer interrogates that comparison in plain English, decides per line, and the decision becomes supplier letters, an order handoff and an audit trail.

What I've chosen not to build

- **The email loop — outreach, replies, and pulling attachments out of an inbox.** The biggest deliberate omission. Wiring a mailbox is a solved problem, and it would have consumed the time that went into the part that is actually hard: deciding what the system should refuse to say. Sending is simulated and labelled as such on every screen. **Reading attachments is fully real** — only the transport is stubbed.
- **ERP and PO submission, payment, invoice matching, goods receipt.** The handoff is a document of record, already structured for whatever consumes it next.
- **Supplier portal, logins, permissions.** Forcing suppliers into my UI is the exact behaviour I set out to avoid.
- **A weighted best-value score.** Asked for, deliberately declined — 87.3 tells you nothing about why a supplier won. Best value is *the cheapest quote from a supplier that meets an explicit, buyer-set bar*: a rule you can argue with.
- **Splitting one line across suppliers, and OCR for scanned PDFs.** The first is unrepresentable by design; the second is reported unsupported rather than guessed at.

System architecture — and why the output is grounded

Every row is a real stage in the running system. The two right-hand columns are the argument: the model is confined to interpretation, and everything it produces is schema-validated and then checked against the source before it reaches a screen.

#	WHAT HAPPENS	WHO	WHAT KEEPS IT HONEST
1	Buyer describes a need Co-pilot asks only what changes a price.	AI	A stated fact must appear verbatim in the buyer's own words, or it is downgraded to a visible suggestion.
2	RFQ is assembled 30 lines, questionnaire, terms, readiness.	Code	Provenance on every field: buyer-stated, AI-recommended, or missing.
3	Replies arrive in any format xlsx · pdf · docx · email · phone photo.	Code	A scanned PDF with no text layer is reported unsupported, never guessed at.
4	Each reply is read One structured extraction call per response.	AI	Instruction-shaped supplier text is stripped before it reaches a prompt. Values from a photo are capped at 75% confidence.
5	Lines are matched Supplier rows mapped to the buyer's lines.	Code + AI	An uncertain match becomes Needs Review , never a silent guess.
6	Prices are normalised Per-1,000 → per piece; discount, MOQ, FX.	Code	No currency named, or a basis that cannot be reduced → held out of the comparison entirely.
7	One comparison A cell per line per supplier, evidence attached.	Code	<i>Not quoted, no response and unresolved</i> are three distinct states. None of them is zero.
8	Buyer asks questions Plain English over the whole dataset.	AI + Code	The model turns the question into a query and describes the result. It never sees a price while planning, and never calculates.
9	Award is proposed Cheapest and best value per line.	Code	A claimed certificate never counts as verified. The buyer overrides freely, with a recorded reason.
10	Decision is executed Letters, order handoff, append-only audit trail.	AI + Code	The letter schema has no numeric field. Every draft — and every buyer edit — is re-checked against that supplier's own data.

Evaluation. 565 automated tests, no network and no model calls, about 30 seconds — the CLI boundary and its failure modes, the trust guards, extraction, normalisation, matching, the comparison dataset, analyst calculations, award validation and

execution, plus one end-to-end test carrying a single RFQ through every phase. Live scripts run the real model against the real fixtures per phase; the award script fingerprints the upstream tables before and after to prove an award changes nothing it should not, and includes a deliberate cross-supplier leak attempt. A built-in extraction playground answers “is any of this hardcoded?” on demand: paste any supplier reply and watch the same pipeline read it live.

Constraints kept

The model must not become the database	Claude turns a question into a structured query and describes the result. Retrieval, filtering, ranking and totals are Python over SQLite.	The analyst can misread what you meant and still cannot be wrong about the number.
Suppliers reply however they like	No template is ever sent; five document readers feed one extraction pass per reply.	Five companies don't have to change how they work.
Uncertainty is shown, not resolved	A price that cannot be reduced to a comparable figure is held out rather than estimated. No fallback exchange rate exists.	The buyer sometimes sees a gap. That is the honest answer.
No API key	The app shells out to the Claude Code CLI on the user's own subscription.	Runs anywhere after one sign-in, but CLI latency pushed the seven most useful analyst questions to be pre-built and instant.

Guardrails

The right-hand column above is the guardrail layer. Three properties of it are worth stating separately, because they are what make the rest trustworthy:

- **A failed check discards, it never repairs.** A supplier letter that invents a figure is thrown away whole and replaced by a deterministic one — there is no path where the model gets to patch its own output into something plausible.
- **Nothing is inferred to fill a gap.** No fallback exchange rate exists anywhere; “\$” is deliberately unmapped because it could be four currencies; an unstated term reads *Not provided*, never a default.
- **Everything is logged.** Every AI call with its prompt, timing and schema result; every award state change with what it replaced. The buyer can inspect both in the app.

What I've learnt

Extraction was not the hard part; it worked early. Almost all the difficulty was in deciding **what the system should refuse to say** — and the sharpest lesson came from my own screen. An early version of the award page recommended a supplier, then blocked the award because that supplier's certificate was unverified. The product was arguing with itself. The fix was not technical: a buyer reaching the award screen has already done the reviewing, so the system should report rather than gate.

The more interesting problem is one layer down. Everyone frames this as document extraction. It isn't. Procurement comparability is a *judgement* — minimum order, price basis, validity, certification — and today it lives only in a buyer's head, which is why it cannot be audited, delegated or checked six months later. Making it explicit and arguable is worth more than reading another PDF format. The most valuable moment in the demo is not the comparison table — it is one question reducing a 30-line answer to 11, because only one certificate is actually on file.

What's next

Inbound email ingestion, feeding the pipeline that already exists. Buyer-tunable comparability rules — today the quality bar is one toggle, and it wants to be a small, named policy a category team agrees once and applies to every RFQ. And a diff view across supplier revisions, since the revision chain is already stored and unexploited.

AI should remove the spreadsheet work, not the buyer from the decision — and where the data will not support an answer, the honest move is to show the gap.